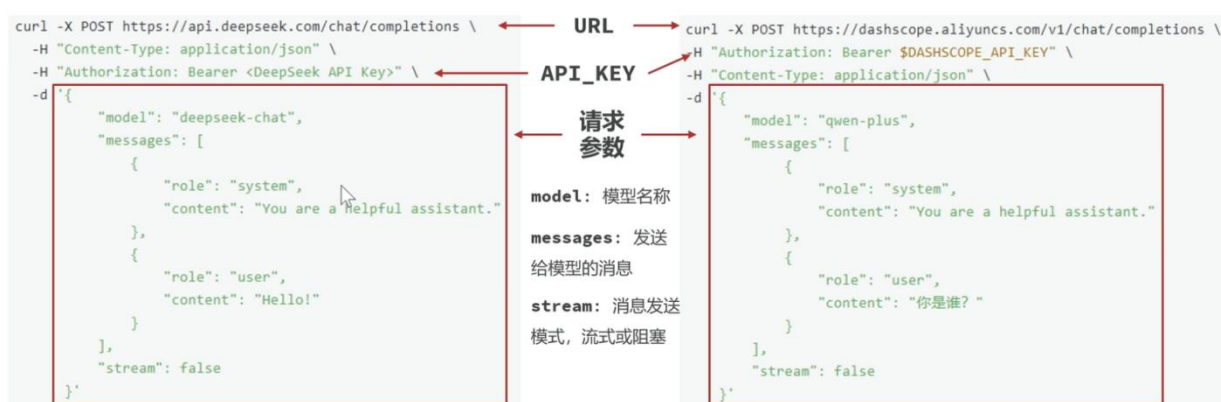


Agent 逻辑梳理

第一章 Agent

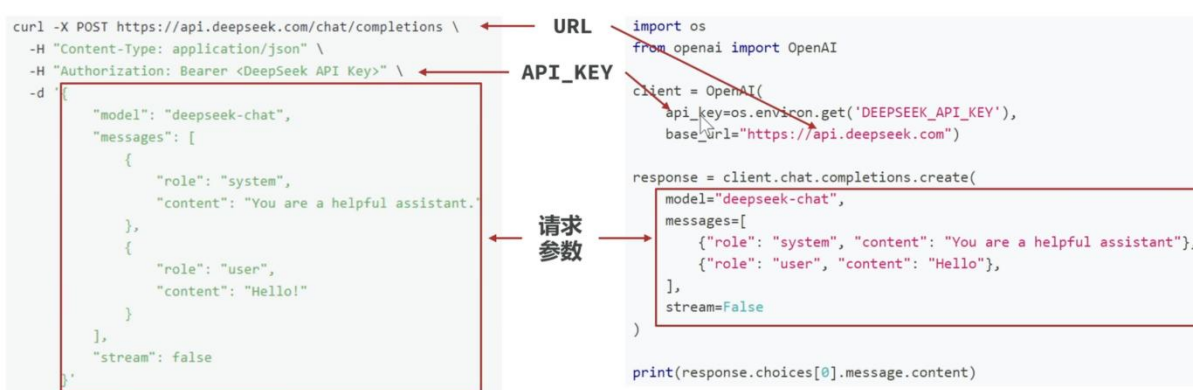
1.1 curl 模型交互

1. 通过 curl 与大模型的 http 接口交互:



2. 通过 openAI SDK 与大模型交互:

OpenAI公司最早提供了调用模型的API接口和对应的SDK，现在的模型基本都支持这套SDK



1.2 上下文管理

1. 上下文管理的核心是：Message 数组的管理，在多轮对话的场景下。
2. 大模型返回的信息作为数组的一项再次注入到 message 数组中
3. 将用户最后的问题在拼接数组末尾

1.3 Agent 核心逻辑

LLM 负责推理，websearch 负责联网搜索，rag 负责本地搜索，skill 负责输出的结构。

本质上是一个搜索引擎的加强版，把一个 Query 由搜索引擎搜出来的结果进行进一步整理。

1.3.1 核心流程

1. 核心流程： 任务规划（大模型完成推理/分析）-> 大模型调用工具（基于工具集）<---->感知反馈（observation）
2. 基本步骤：

Agent 执行流程:

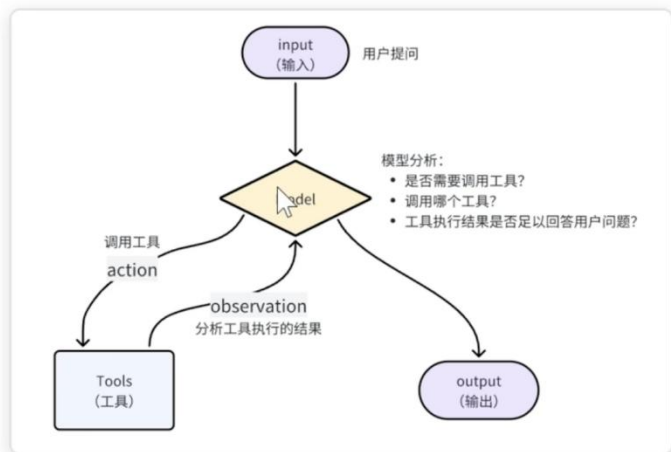
1 用户提问 (Input)

2 模型分析 (Reasoning) : 分析需求, 规划任务, 判断是否需要调用工具, 调用哪个工具

3 调用工具 (Action)

4 感知分析 (Observation) : 分析工具返回结果, 判断是否足以完成目标, 不足则继续前面步骤

5 生成结果 (Output)



1.3.2 Agent 设计模式

1. SimpleAgent:

- * 流程: 直接输入 → 直接输出 (无思考链)
- * 特点: 没有推理、不调用工具、不自我修正
- * 应用场景: 问答、简单文本生成

2. ReActAgent: (边想边查资料)

- * 流程: 一边思考一边行动, Reason + Act;
思考 -> 行动 -> 反馈 -> 思考 -> 答案/再次行动
 - * 特点: 有 CoT、能调用工具 (API / DB / 搜索)、动态决策
- 下一步
- * 示例:

Q: 今天上海天气?

Thought: 我需要查询天气

Action: 调用天气API

Observation: 25°C 晴

Thought: 已获取信息

Answer: 今天25°C, 晴

3. ReflectionAgent: (写完作业会检查)

- * 流程: 行动 -> 检查 -> 优化

- * 特点: 引入“自我评估”、提高准确率、常用于复杂任务 (写作 / 编程)

- * 示例: 编程场景

Step1: 给出答案

Step2: 自己批评:

- 有没有错误?
- 有没有遗漏?

Step3: 修改答案

4. PlanAndSolveAgent: (先写提纲再写作文)

- * 流程: 计划 -> 执行 1 -> 执行 2 ->...->答案

- * 特点: 明确任务拆解

- * 示例: Claude Code 调用 Agent 方法时

Q: 做一个GOOG估值分析

Plan:

1. 收集财务数据
2. 建立DCF模型
3. 输出估值

Execute:

Step1 → Step2 → Step3

5. 进化流程:

```
Simple
  ↓
ReAct (能用工具)
  ↓
Reflection (会检查)
  ↓
Plan&Solve (会规划)
```

6. 多阶段 Agent: Plan + ReAct + Reflection

- 1 Plan (拆任务)
- 2 ReAct (查数据 / 调API)
- 3 Reflection (检查逻辑)

👉 这就是现在主流的“多阶段Agent”

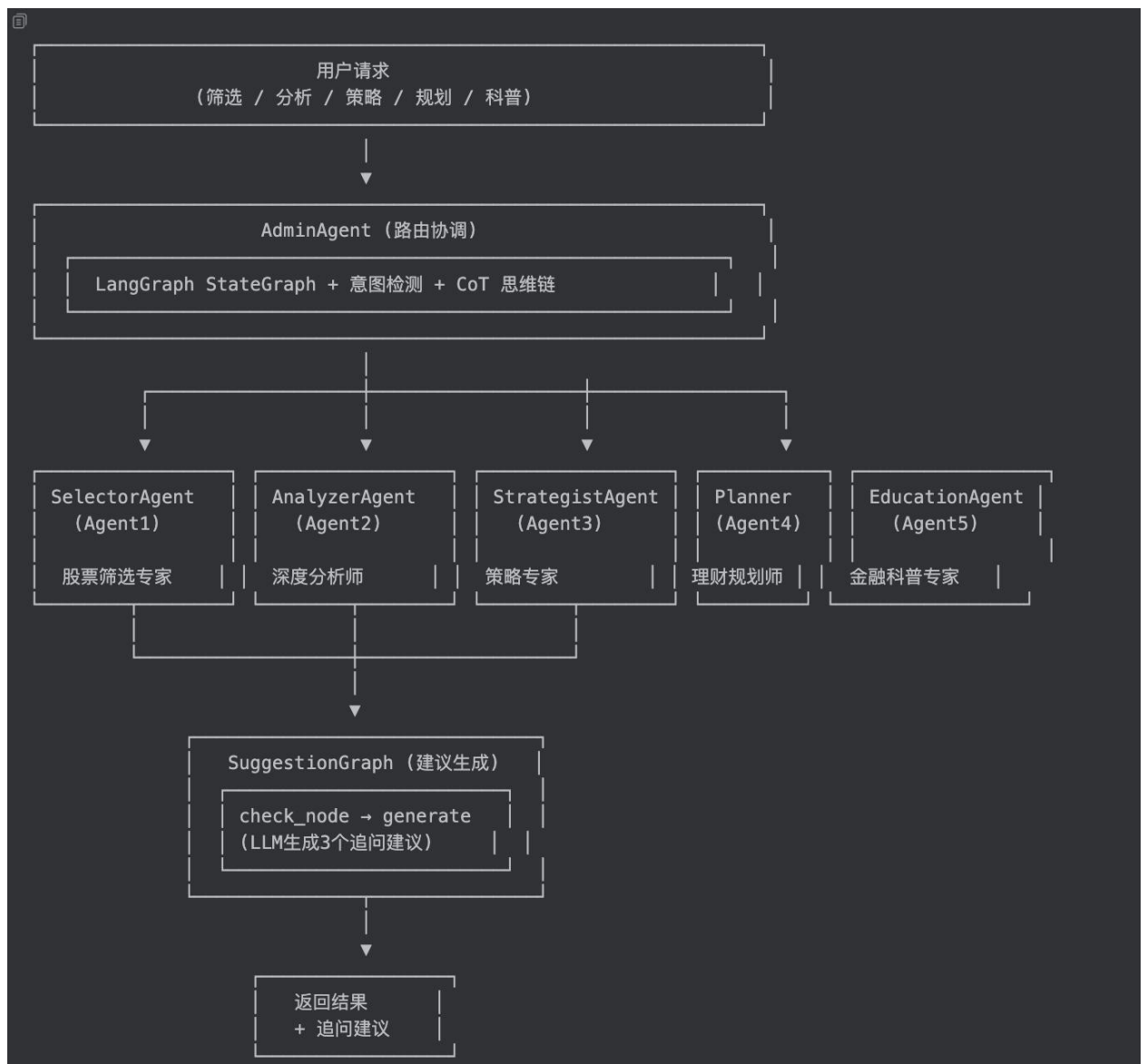
1.3.2.1 多智能体系统 (Multi-Agent System)

1. 基于 LangGraph 实现 AdminAgent 编排的多智能体协同工作模式
2. 各专业 Agent 负责不同的投资研究任务，通过 AdminAgent 统一调度
3. AdminAgent 的调度机制: One-shot 调度

Agent 类型

Agent	编号	功能	触发关键词
SelectorAgent	1	股票筛选	筛选、选股、找股票、有哪些
AnalyzerAgent	2	深度分析	分析、研报、怎么样、值得买吗
StrategistAgent	3	策略生成	策略、定投、仓位、配置
PlannerAgent	4	理财规划	理财规划、资产配置、退休
EducationAgent	5	金融科普	什么是、解释、科普、概念

4.MultiAgent 架构:



4. AdminAgent:

意图识别 - 理解用户想要什么

路由决策 - 确定调用哪个 Agent

流程编排 - 管理 Agent 间的执行顺序

结果整合 - 聚合最终响应

1) LLM 意图识别决策机制:

```
DECISION: <selector|analyzer|strategist|planner|education|general>  
QUERY_TYPE: <timeliness|verification|static>  
NEED_WEB_SEARCH: <yes|no>  
STOCKS: <股票代码列表, 逗号分隔>  
REASON: <决策原因>
```

1.4 Function Call

1.4.1 大模型常用参数

入参:

1. modal
2. message
3. tools

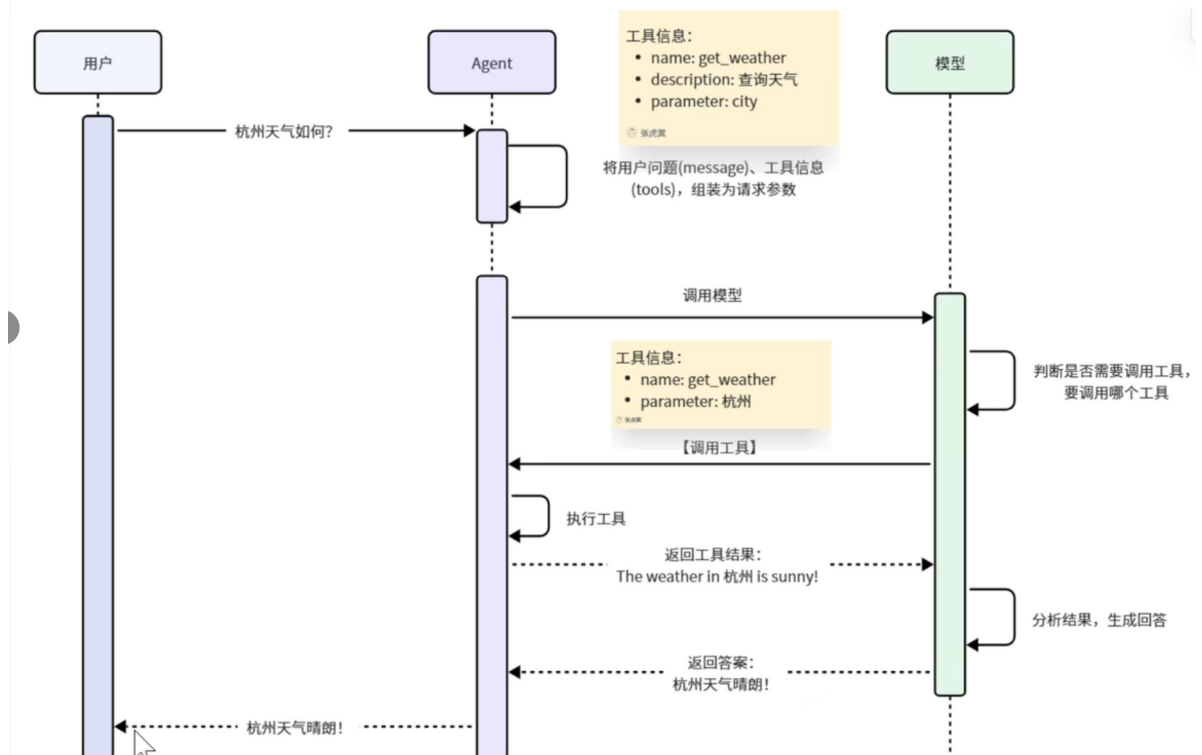
4. stream
5. temperature
6. top-p/top-k

出参:

1. role
2. content
3. tool_calls: function name、function params

1.4.2 工具调用步骤

- a. 注册工具
- b. 将工具描述注入 prompt, langchain 中会封装到框架中
- c. 让大模型进行推理和工具选择
- d. Agent 执行 tools
- e. 将结果再次返回给大模型
- f. 大模型再次判断返回结果还是继续调用 tools



1.4.3 tool use

python

 Copy  Download

1. 定义函数

```
functions = [{
    "name": "add_numbers",
    "description": "计算两数之和",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number"},
            "b": {"type": "number"}
        }
    }
}]
```

2. 用户输入

```
user_input = "请计算 35 加 78"
```

3. 第一次调用 LLM

```
response = llm.chat(messages=[user_input], tools=functions)
```

4. 检查是否有 tool calls

```
if response.tool_calls:
    call = response.tool_calls[0]
    if call.function.name == "add_numbers":
        args = json.loads(call.function.arguments)
        result = add_numbers(args["a"], args["b"]) # 真实计算 113
```

5. 将结果回传

```
response2 = llm.chat(messages=[
    user_input,
    response,
    {
        "role": "tool",
        "tool_call_id": call.id,
        "content": str(result)
    }
])
print(response2.content) # 输出: "35 加 78 等于 113。"
```



1.4.4 finAgent 中的 call tool 实现

```
Agent.ainvoke()
├── 1. 问题分析
│   └── AgentCallChain.log_call("问题分析")
├── 2. 数据检索规划
│   └── tools_plan = await _plan_data_retrieval(message)
│       ├── registry.get_tools_prompt() → 生成工具列表
│       └── LLM 分析 → ["snapshot", "kline", "buffett"]
├── 3. 数据获取
│   └── for tool in tools_plan:
│       ├── "article"
│       │   └── get_local_rag_client().search_knowledge()
│       ├── "buffett"/"wangchuan"
│       │   └── self.get_framework(tool)
│       ├── "news"
│       │   └── get_news_client().get_news_summary()
│       └── "snapshot"/"kline"/...
│           └── get_futu_api_client().{method}()
└── 4. 结果聚合
    └── _generate_xxx() 整合数据生成回答
```

	这个项目	Function Call
工具选择	LLM 在 prompt 中选择	LLM 直接决定
工具执行	代码手动 if/elif 路由	SDK 自动执行
扩展方式	修改 _call_tool()	定义 schema 自动映射

这个项目用的是 "LLM 辅助的工具规划 + 手动路由执行"，而不是 "LLM Function Call"。

1.5 WebSearch

1.数据源:

- 1.1 搜索引擎供应商提供的 api 已经凉了 (google 和 baidu 已经不提供此类型的 API , bing 官方搜索api去年11月下线)。
- 2.2 爬取搜索引擎结果的 SERP(search engine result page) API 的供应商, 如果 serpAPI 、 serper 、 searchAPI 等。
- 3.3 自建搜索API的, 现在也大都和 AI 集成的比较丝滑, EXA 、 Tavily 、 Brave 、 Linkup 等。
- 4.4 用爬虫技术发起搜索引擎请求的工具, 如 Duckduckgo MCP 、 orz-mcp 等。
- 5.5 AI模型自带搜索功能 openai anthropic perplexity 等。

3. 执行逻辑:

检索 -> 增强 -> 生成

检索: DuckDuckGo、腾讯新闻 CLI、Static RAG、FuTuAPI

增强: 实效性评价、权威性评价、排序

生成: Prompt->LLM

1.5 Agent 幻觉及解决方案

大模型的局限性:

知识局限性

知识实时性

幻觉问题

1.6 MCP

1.7

第二章 Langchain

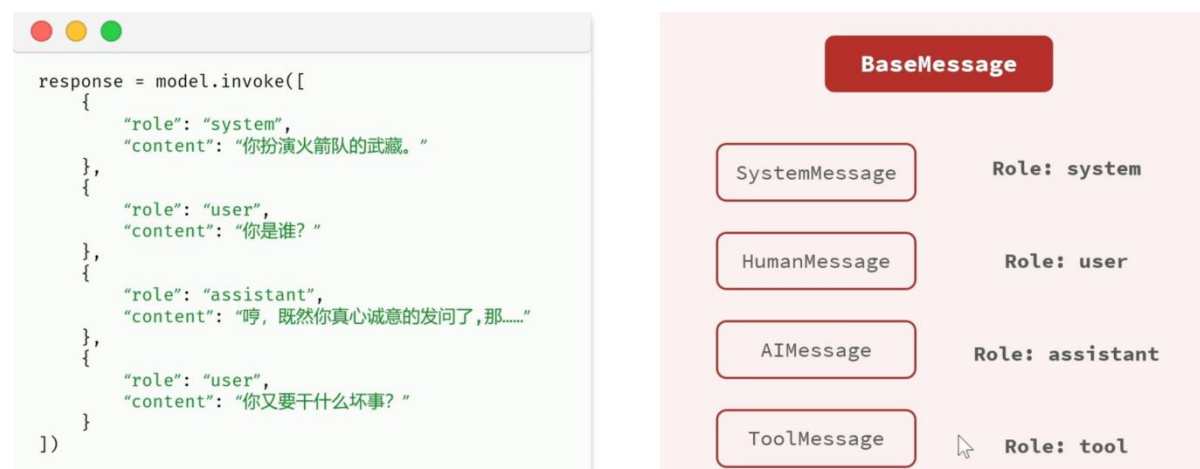
2.1 基本开发步骤

2.1.1 依赖安装

- 安装 uv
- 安装 Langchain、安装模型调用的 sdk、环境变量配置、
- 创建模型、模型配置、模型调用

2.1.2 Message

2.1.2.1 消息类型



一般来说，系统提示词（System Prompt）会包含以下几个部分，通常按此顺序排列：

- **身份角色（Identity）**：描述AI的职责、沟通风格和总体目标。
- **指令说明（Instructions）**：请指导模型如何生成所需的响应。它应该遵循哪些规则？模型应该做什么，以及模型绝对不能做什么？
- **对话示例（Examples）**：提供可能的输入示例，以及模型期望的输出。
- **背景信息（Context）**：向模型提供生成响应所需的任何额外信息，例如RAG的额外知识库数据，或您认为特别相关的任何其他数据。

2.1.2.2 多模态消息

1. 包含图片、音频、视频的消息，图片，base64 格式传给模型，但会存在过载的情况。
2. 多模态模型默认支持获取图片 URL，结合 OSS 方案+缓存 (TODO: 待验证)

2.1.3 提示词工程

2.1.3.1 系统提示词

```
system_prompt = """
# 身份
- 你是一个科幻作家，根据用户的要求创建一个太空之都。

# 指令
- 请务必以JSON格式输出，不要加任何markdown样式。

# 示例:
user: 月球的首都是什么?
assistant:
{
  "name": "月华市 (Lunaria)",
  "location": "位于月球正面赤道附近的静海基地遗址之上，依托巨大的穹顶与地下网络建成",
  "vibe": "冷冽、高效、革新",
  "economy": "氦-3能源开采、量子通信枢纽、尖端生物圈农业"
}
"""
```

提示词是调用模型的桥梁，沟通 Agent 数据、规则等内容的中间件。

由工具产生的数据通过提示词和 LLM 沟通

根据不同场景、不同用户可以生成不同的提示词。

Agent 提示词:

- 1、告诉 Agent 它的角色

2、告诉 Agent 它的触发条件与决策规则：关键词、触发条件

3、告诉 Agent 它的边界：重要信息、必须、绝对、禁止

上下文记忆：

2.1.3.2 SKILL

skill 是提示词的进阶版本，skill 中定义 interface，Agent 在执行的时候会通过 bash 生成代码，

2.3.3 工具

2.3.4.1 通过 Langchain 定义工具

方案一：

1.2.使用函数名和文档注释描述工具

如果不@tool装饰器没有定义工具名和作用描述，此时：

- 工具名：默认就是函数名
- 工具所需的参数：默认就是函数的参数列表
- 工具作用的描述：默认就是函数的文档注释

```
1 from langchain_core.tools import tool
2 # 通过tool装饰器定义工具
3 @tool
4 def square_root(x: float) -> float:
5     """Calculate the square root of a number"""
6     return x ** 0.5
✓ [3] < 10 ms
```

```
1 # 定义一个查询天气的tool
```

方案二：

1.3.定义Pydantic Model描述参数

如果函数的参数比较多，而且比较复杂，此时建议通过pydantic model来描述参数列表。

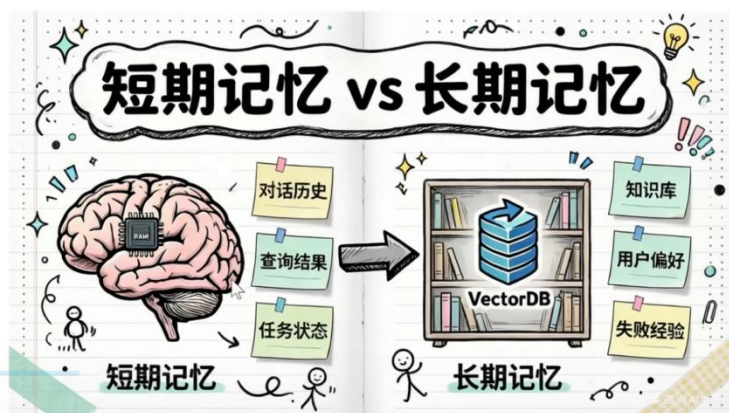
```
1 # 通过自定义model来约束入参
2 from pydantic import BaseModel, Field
3 from typing import Literal
4
5
6 # 例如一个查询天气的tool
7 class WeatherInput(BaseModel):
8     """ 查询天气的输入参数。 """
9     location: str = Field(description="City name or coordinates")
10     units: Literal["celsius", "fahrenheit"] = Field(
11         default="celsius",
12         description="Temperature unit preference"
13     )
14     include_forecast: bool = Field(
15         default=False,
16         description="Include 5-day forecast"
17     )
```

2.3.4.2 框架已经集成的工具

2.3.4 Agent 记忆

Agent的记忆 (Memory) 分两类：

- 短期记忆 (short-term memory)：当前任务或会话的上下文
- 长期记忆 (long-term memory)：跨任务或会话的经验与知识



2.3.5.1 短期记忆 (Short-term Memory)

1. 应用场景：对话历史、查询结果、任务状态
2. 内存：短期记忆作用域：当前任务或会话的上下文
3. 数据库存储：
 - (1) 避免多轮对话导致内存溢出：基于 Langchain AgentState 实现 | 基于数据库的持久化存储 SQLite(langchain-checkpoint-sqlite)

(2) 模型上下文溢出的问题：上下文记忆管理：



① 用另外一个 AI 模型形成上下文总结摘要，用摘要替换原本的上下文内容

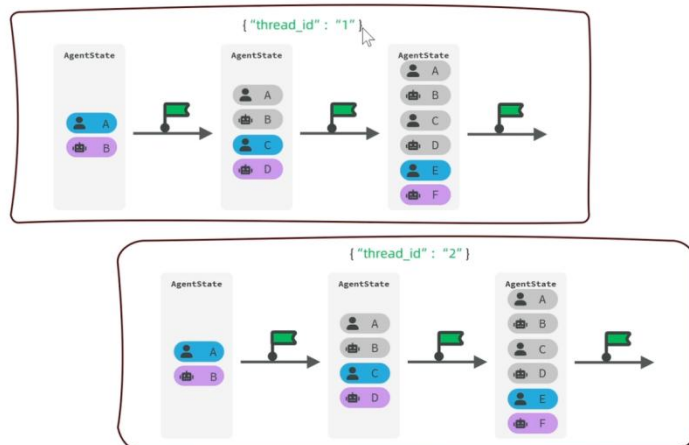
② 同时对快照进行压缩

4. 使用 SummarizationMiddleware 实现摘要

(1) 触发条件：context size\ Absolute token\ message count

5. 记忆存储 checkpointer、time traveling:

LangChain提供了**Checkpoint**对象来保存AgentState，每一次用户与AI的交互都会生成一个快照，记录为一个checkpoint。同一会话的多个checkpoint形成一个组，用同一个**thread_id**来标记。



6. VectorDB (TODO)

2.3.5.2 长期记忆 (Long-Term Memory)



2.3.5.3 用户意图识别

1.AdminAgent 内部路由逻辑:

- 预检查:

-如果消息包含关键词，直接定向路由

- 特殊功能: 导出报告关键词, 直接执行程序
- 上下文构建: 从 history 中提取最近 6 条消息作为上下文, 附加当前 stocks 信息
- LLM 决策: 调用 `create_message_with_fallback` 让 LLM 决定路由目标

1. Context 上下文提取:

(1) 问题: CoT 中 ConText 关联的股票信息与 query 关联性低:

优化: stock:路由时优先取: 请求中的 stocks (`chat_request.stocks`)

→ `_extract_stocks_from_message(Short Memory)` →

`_extract_stocks_from_history(Long Memory)`

(2) 问题: `profiles.json` 每次分析都可能触发 3+ 次写盘

(`set_current_stocks` + `add_to_watchlist` + `save_analysis`), 无批量、无合并, 频繁 IO

优化: `set_current_stocks` 动态更新:

1. 新查询的股票插入到最前面 (最新优先)
2. 从现有列表中移除已在本次查询中的股票 (避免重复)
3. 最多保留 3 只股票
4. 旧股票不在新查询中则被淘汰

批量落盘: 请求结束时调用 `memory_mgr.long_term.flush()`

方案	优点	缺点
当前：持久化到 profiles.json	跨进程/重启保留	有IO开销
内存缓存（带TTL）	快速	重启丢失，新session无效
只存最近一次对话的session	不需额外存储	需查checkpoint db

(3) 问题：响应速度慢 -- LLM 和硬编码的取舍

优化：快速通道分流：Query 和 Agent 的映射明晰-硬编码

(4) 问题：推理可能提问 suggestion-graph 相关度不高

(5) 问题：对 Query 的判断不准确，route_node 出现幻觉

优化：关键词匹配直接路由 -》LLM 推理路由

1.优化关键词：增加专业术语关键词匹配

2.优化 prompt：增加限制和更多说明

2.2.5 Multi-Agent 调度系统

2.2.2.1 Agent 调度系统

1. 实现意图检测，路由到正确的 Agent
2. 使用 WorkflowManager 管理状态转换
3. 支持完整流程：筛选 → 分析 → 策略
4. 支持研报后的建议模板（LangGraph）

实现路径：

- SYSTEM - 系统初始化
- CHAT_API - API 入口收到请求
- DISPATCHER - 调度器接收和路由

- INTENT_DETECTION - 意图检测结果
- WORKFLOW - workflow状态变化
- SelectorAgent / AnalyzerAgent / StrategistAgent - 各 Agent 调用
- SuggestionGraph - 建议模板图

2.2.6 Suggestion Graph

流程：

```

用户提问 → Agent处理 → 返回结果 → 触发get_suggestions()
                                     ↓
                               SuggestionState 构建
                                     ↓
                               check_node: 判断是否需要建议
                                     ↓
                               generate_suggestions: 调用LLM生成问题
  
```

2.2.2.2 Agent 与技术方案

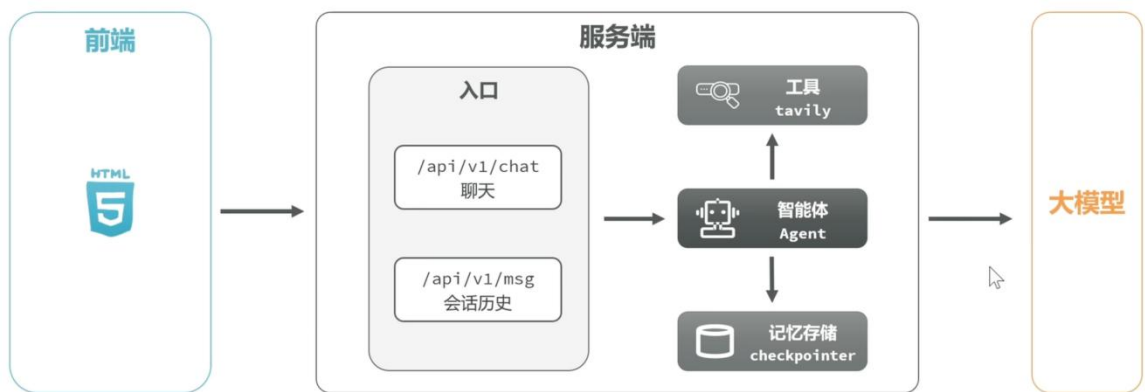
1. 关键词预检测 → 直接路由（最快路径）
2. 都没匹配 → 使用 LLM 进行决策（后备）

```

462 +根据用户意图，返回你的调度决策。
463 +
464 +分析用户消息：
465 +- "分析今天市场的基本情况" 明显是要求分析全球市场/大盘，应该路由到 market_analyzer
466 +- "帮我看看腾讯" 是股票分析，应该路由到 stock_analyzer
467 +- "比特币现在多少" 是加密货币分析，应该路由到 crypto_analyzer
468 +
469 +返回格式：
470 +DECISION: <selector|stock_analyzer|crypto_analyzer|market_analyzer|planner|education|general>
471 +REASON: <简短决策原因>""
472 +
  
```

2.2 服务搭建

2.2.1 Agent 架构



第三章 RAG

3.1 基本流程

RAG

Retrieval Augmented Generation



检索



增强



生成



1. 文档预处理:

(1) 文档检索 (doc、docx、pdf) - 排版分析/复杂表格

(2) 表格检索 (excel csv) - 长表格大量上下文 token

(3) 数据库检索 (mySql、oracle) - SQL 生成不准确

2. Chunking-分片:

(1) 按字数 - 定长切割

(2) 按段落 | 按章节 - 颗粒度大, 每个分片大小不一, 大分片向量化可能不准确

(3) 语意切割 - 颗粒度细, 检索难度大

3. Embedding: 词向量

4. IndexDB-向量数据库

5. Indexing-索引

6. 检索: Query 和 词向量计算相似度 (相似度计算 | 欧氏距离)

(1) 全文检索: 关键词匹配 - BM25 分值

① 常见的基于词袋的关键词检索评分

② 对 TF-IDF 改进, 引入词频饱和、文档长度归一化

(2) 向量检索: 近义词匹配 - 向量相似度分值

7. 召回:

(1) 多路召回算法: RRF (倒数排序融合)

- 必须保证所有参与排序的算法都尽可能准确

- 排名越靠前 (rank 越小), 倒数越大, 贡献越高。

-

公式

对于物品 d ，其在 n 个排序列表中的融合分数为：

$$\text{score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

- $\text{rank}_i(d)$ ：物品 d 在第 i 个列表中的排名（从1开始）
- k ：平滑常数（通常取 **60**，目的是削弱高排名物品的绝对优势）

最终按分数**降序**排列得到融合后的列表。

直观理解

- 排名越靠前（rank 越小），倒数越大，贡献越高。
- 常数 k 避免某一列表中的第一名（贡献约1/61）相比另一列表的第十名（贡献约1/70）差距过大，让不同列表的贡献更平滑。

顺序	全文检索	向量检索	RRF
1	d1	d4	d4
2	d4	d3	d1
3	d3	d2	d3
4	-	-	d2

RRF的

$$\begin{aligned}d_1 &= \frac{1}{1} + 0 = 1 \\d_2 &= 0 + \frac{1}{3} = 0.33 \\d_3 &= \frac{1}{3} + \frac{1}{2} = 0.83 \\d_4 &= \frac{1}{2} + \frac{1}{1} = 1.5\end{aligned}$$

(2) 精准重排序：基于 RRF 的排序结果重新排序


```

# 输入
query = "什么是机器学习? "
documents = [
    "机器学习是人工智能的一个分支...",
    "今天天气很好。",
    "深度学习是机器学习的一个子领域..."
]

# 输出 (模型返回每个文档的得分)
scores = [0.95, 0.03, 0.88]

# 或者直接返回排序后的文档列表
reranked_docs = [
    "机器学习是人工智能的一个分支...", # score 0.95
    "深度学习是机器学习的一个子领域...", # score 0.88
    "今天天气很好。" # score 0.03
]

```

(3) 选出 TopK 个相似的词

示例流程

场景：短视频推荐

- 多路召回：
 - 协同过滤召回 200 条
 - 双塔向量召回 300 条
 - 热点召回 100 条
 - 合并去重后得到 500 个候选视频
- 重排模型 (如 DeepFM)：

对每个候选视频提取：用户历史点击序列 embedding、视频类目/作者特征、实时交互统计 (如过去 1 小时点赞率)

模型输出一个 0~1 的点击概率分

按该分降序排列，取 top 50 推送给用户

8. 增强：Prompt：将 TopK 个词作为上下文和 Context 进行打包

9. 生成：大模型根据指令生成

3.2 检索算法

1. 词袋模型 + TF-IDF (向量化后计算余弦相似度) : 只能捕捉词频信息, 基本丢失了词序和深层语义。
2. 余弦相似度: 现代 RAG 要的是句子级或段落级的语义匹配 (例如“猫坐在垫子上” 和 “一只猫在垫子上休息” 应该很相似), 这必须由模型把整个 chunk 里的词组合起来理解, 才能做到。对 Chunk 的大小严格要求; 准确度依然不足
3. 欧式距离:
4. Embedding 算法:

3.3 检索优化

1. 文本内容整理: 文档转文本的兼容差异
 2. Chunk 切片尺寸确定: 太大了浪费 token, 太小了语意无法理解
 3. 对检索的 query 清洗后, 再进行向量化
 4. 检索准确率: 混合检索 (关键词查找 + 向量查找) =》 Rerank (重排模型)
 5. 用户提问的模糊性: Query Rewrite (查询重写) LLM 结合上下文重写 Query
 6. 基于 BM25+向量检索的优化
- 基本条件: 更少的 token 更好的效果

基本思路：检索 - 增强 - 生成

优化思路：检索结果优化

目前方案：

RAG：基于文本片段

WebSearch：基于文本摘要

SKill+grep 或 TF-IDF 优化：基于文件

3.4 reRank 模型

3.4.1 BM25

1. Best Matching 25, 一种经典的词袋 (Bag-of-Words) 检索模型。它不依赖向量或深度学习，完全基于词语的统计信息。

(1) 查询词在文档中出现的频率 (TF) - 出现次数越多越相关；

(2) 查询词的区分度 (IDF) - 在所有文档中越稀有，贡献越大；

(3) 文档长度的影响 - 文档过长会“稀释”词频，需要适当惩罚。

2. BoW：只关注词频 TF

3. TF-IDF：它通常作用于词袋的“词频”上，用逆文档频率 (IDF)

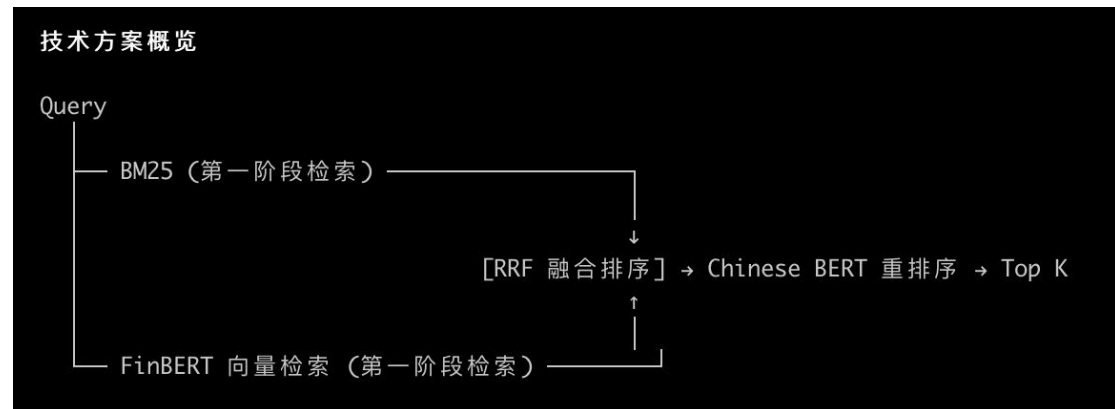
来调整权重: $TF-IDF = TF \times IDF$ 。

3.4.2 Vector

1. 存储方案:

PostgreSQL-pgVector

2. 整体流程:



3. 模型:

- (1) FinBERT: 单一语种 (主要为中/英) 的金融领域专用模型, 适用金融文本分析, 如金融情感分析、ESG 分类、新闻解读等。定位是一个金融专家, 为了让模型更懂“金融行话”, FinBERT 在 BERT 的基础上, 使用了财报、研报、新闻等海量金融文本进行“继续训练”。
- (2) FinBERT2: 它在继承了上一代优势的同时, 进一步引入了规模更大、质量更高的金融语料进行预训练
- (3) mBERT: 多语种支持
- (4) Qwen: 泛化性更强, 基于大语言模型 (Decoder/Encoder-Decoder) 的专用向量模型

3.4.3 ReRank 的必要性

Embedding:

- 语意模糊、信息丢失, 召回高, 算法是字面意思匹配, 准确度不行。
- Bi-Encoder (双向编码器): 余弦相似度计算, 查询速度快

ReRank:

- 用于补足 Embedding 缺陷, 在高召回的信息中, 提高准确度。
- 准确度高, 召回差, 直接用于检索非常浪费时间和成本, 且效果不好。
- 如果没有 Rerank, 直接将 embedding 结果投喂给大模型, 会导致大模型的准确度下降。
- Cross-Encoder: 速度慢, 开销大

3.4.3 RRF

3.4.4 伪标签策略

3.4.5 模型输入输出

3.3 模型评估标准

1. 逻辑推理能力 (Logical Reasoning)
2. 指令遵循能力 (Instruction Following)
3. 防幻觉能力 (Anti-Hallucination)

3.4 框架及结构





1.GraphRAG

2.AgenticRAG



第四章 WebSearch MCP

3.1 响应精度调优

当前架构

当前的 `ainvoke` 流程是：
`ainvoke()`
→ `search_engine.parallel_search()` # 预先获取数据
→ `_generate_analysis(message, search_context)` # 把数据注入 prompt
→ LLM 生成回答（无工具调用能力）

3.2 信息检索主要流程（旧）

query → need_web_search → web_search_node → 并行搜索 → 结果聚合 → search_context

3.2.1 need_web_search:

类型	触发特征	是否联网
timeliness	今天、最新、当前	✓
verification	是否、证实、突破	✓
static	概念、知识性	✗

3.2.2 WebSearch 核心组件:

组件	文件	职责
SearchEngine	core/search_engine.py	并行搜索多数据源
ResultAggregator	core/result_aggregator.py	结果去重、评分、格式化
web_search_node	core/admin_agent.py	LangGraph 节点

SearchEngine（并行搜索）：

数据源	判断函数	触发关键词示例
FutuAPI	LLM 规划	有股票代码时自动分析
DuckDuckGo	无(始终)	-
腾讯新闻	query_type	timeliness/verification
Static RAG	_is_investment_query()	投资、巴菲特、护城河、股东信、ROE、苹果
Finnhub	_is_us_stock_query()	美股、nasdaq、苹果、微软、英伟达、特斯拉
CoinMarketCap	_is_crypto_query()	比特币、eth、以太坊、加密货币、bnb、sol
CoinGecko	_is_crypto_query()	同上
Sina Gold	_is_gold_query()	黄金、gold、xau、金价、comex
BitSoulStock	_is_astock_query()	A股相关（委托 mcp_client）

ResultAggregator（结果聚合）：

- Process:处理结果并返回格式化文本

6. 格式化输出 `_format_context()`

输出格式：
联网搜索结果

****时效性信息**** (最新优先):

[1] 标题 [来源 | 2026-05-19]
内容摘要 ...
来源: URL

[2] 标题 [来源 | 2026-05-18]
...

****重要引用规则****:

- 1. 必须使用 [编号] 格式标注所有引用的来源
- 2. 每个事实陈述后必须跟相应的来源标注
- 3. 例如: 比特币今日价格为 \$95,000 [1]
- 4. 例如: 根据 [2] 显示, 市场流动性充足
- 5. 如果没有引用来源的陈述, 需明确说明 "无明确来源"

- 处理流程:

步骤	方法	说明
1	<code>_deduplicate()</code>	URL 去重
2	<code>_apply_timeliness_score()</code>	时效性评分
3	权威性调整	$authority_score = score * 0.7 + score * 0.3$
4	排序	按 <code>authority_score</code> 降序
5	<code>_format_context()</code>	格式化为 LLM 上下文

- 时效性评分:

仅对 **timeliness** 和 **verification** 查询生效

发布时间	权重系数
≤ 24小时	1.0（不降权）
≤ 72小时（3天）	0.8
≤ 168小时（7天）	0.5
> 7天	0.2
无时间戳	0.7

- 权威性评分:

数据源	authority_score
Sina Gold	0.95 / 0.90 / 0.75
CoinMarketCap	0.90 / 0.95
CoinGecko	0.90
Finnhub	0.90 / 0.85 / 0.80
BitSoulStock	0.90 / 0.85
AnySearch	0.85
FutuAPI	0.85
Static RAG	0.85
腾讯新闻	0.80
DuckDuckGo	0.60

同一数据源内部，实时行情 > 详情/公司 > 一般列表/新闻:

数据源	返回类型	authority_score
Sina Gold	实时行情	0.95
	市场分析	0.90
	资金流向(无数据时)	0.75
CoinMarketCap	加密货币详情	0.95
	加密货币列表	0.90
Finnhub	实时行情/指数	0.90
	公司概况	0.85
	市场新闻	0.80
BitSoulStock	实时行情/买卖信号	0.90
	排行/基本面/K线	0.85

3.3 MCP 实现

第四章 动态 SKILL Engine

完整执行流程

阶段1: Skill 发现
SkillRegistry.discover_skills()
- 扫描 static/skills/ 目录
- 解析每个目录下的 SKILL.md 提取元数据
- 构建 keyword_index (关键词 → Skill 映射)
- 注册 SkillCard (用于 LLM 系统提示注入)

↓

阶段2: Skill 触发检测
SkillTriggerDetector.detect(query)
1. registry.match_keywords(query) 遍历 keyword_index
2. 计算 confidence 评分
3. 返回匹配最高的 SkillMatch

↓

阶段3: Skill 执行
DynamicSkillEngine.execute_skill(skill_name, params)

阶段3详解: Skill 执行

```
execute_skill()
├── 1→ registry.load_skill_full(skill_name) # 懒加载完整内容
│   └── SkillParser.parse_file() 解析 SKILL.md
├── 2→ 根据 execution.mode 决定执行方式
│   ├── sequential: _execute_sequential()
│   │   └── for step_config in execution.rules:
│   │       ├── executor.execute_script(脚本路径, params)
│   │       └── outputs[output_var] = result["data"]
│   └── parallel: _execute_parallel()
│       └── ThreadPoolExecutor 并发执行所有脚本
└── 3→ _render_output(skill, outputs)
    ├── 有模板: 替换 ${变量名}
    │   └── 支持嵌套: ${var.key}
    └── 无模板: _format_as_markdown()
        └── list of dicts → Markdown 表格
```

第四章 功能设计

Agent 的主要形式:

1. 知识问答类: 豆包
2. 数据看板类: 金融交易看板
3. 执行工具类: claude code
4. Agent 平台: coze, <https://www.astrocade.com/>

4.1 AdminAgent

4.2 基于新闻摘要的 MCP 功能

1. 调用逻辑: 硬编码匹配关键词新闻 =》 调用 MCP

判别模型+Skill => LLM => Reflection => 结果

```
• MCP 触发逻辑在 chat() 函数中:

# api/routes/chat.py:54-56
if _is_news_request(request.message):
    logger.info(f"[DEBUG chat] This is a news request")
    response_content = await _call_news_mcp(stocks, days=7)

触发条件: _is_news_request() 函数检测关键词

# api/routes/chat.py:375-385
def _is_news_request(message: str) -> bool:
    news_keywords = [
        "新闻", "最新消息", "资讯", "动态",
        "获取新闻", "获取最新", "查新闻", "看新闻",
        "最近的新闻", "最新新闻"
    ]
    return any(keyword in message.lower() for keyword in news_keywords)

完整调用链:
用户: "招商银行最新新闻"
↓
chat() → _is_news_request() → True
↓
_call_news_mcp([{"code": "600036", "name": "招商银行"}], days=7)
↓
启动 news_mcp_server.py 子进程
↓
session.call_tool("get_news_summary", {...})
↓
返回新闻摘要
```

web_search_node:

聚合结果保存到 state 中:

```
# 3. 聚合结果 (带 citations)
aggregated = result_aggregator.process_with_citations(results, query_type)
search_context = aggregated["context"]
citations = aggregated["citations"]

# 4. 保存到 state
state["web_search_results"] = search_context
state["web_search_citations"] = citations
```

输入

参数	来源	说明
message	state	用户查询内容
query_type	state	"timeliness" / "verification" / "general"
stocks	state	股票代码列表
need_web_search	state	是否需要联网搜索

输出

字段	说明
state["web_search_results"]	格式化后的搜索上下文
state["web_search_citations"]	引用来源列表

4.3 基于巴菲特的 Rag

4.4 suggetionGraph 的推荐建议

已完成的修改：

1. `core/suggestion_graph.py`
 - 为每种 `agent_type` 定义了独立的 `SUGGESTION_PROMPTS` (`selector/analyzer/strategist/general`)
 - `get_suggestions()` 增加 `agent_type` 参数
 - 删除了不可靠的 `detect_agent_type()` 函数
2. `api/models.py`
 - `ChatResponse` 增加 `agent_type` 字段
3. `core/admin_agent.py`
 - `invoke()` 返回 `{"content": ..., "agent_type": ...}` 字典
4. `api/routes/chat.py`
 - `chat` 接口返回 `agent_type`
 - `FollowupRequest` 增加 `agent_type` 字段
 - `followup` 接口传入 `agent_type` 给 `get_suggestions()`
5. `static/index.html`
 - 添加 `lastAgentType` 变量保存响应中的 `agent_type`
 - `fetchAndDisplaySuggestions()` 接收并传递 `agent_type`

调用流程：

用户发送消息 → `POST /api/chat`
→ `AdminAgent` 返回 `{"content": ..., "agent_type": "analyzer"}`
→ 前端保存 `agent_type`
→ 助手回复完成后调用 `followup`
→ `POST /api/chat/followup {"agent_type": "analyzer", ...}`
→ 根据 `agent_type` 使用对应的 `suggestion prompt`

4.5 CoT 分层推理优化

第五章 Claude.md 强化工具

1. clear 大法
2. 抛出问题，AI 给出答案，梳理逻辑（多轮询问、源码查看），确认答案
3. 修改前与我确认
4. 执行任何命令时，必须在命令前说明该命令的作用
5. 反问问题原因